

练习 6.4.3: 使用图 6-22 所示的翻译方案来翻译下列赋值语句:

2) $x = a[i][j] + b[i][j]$

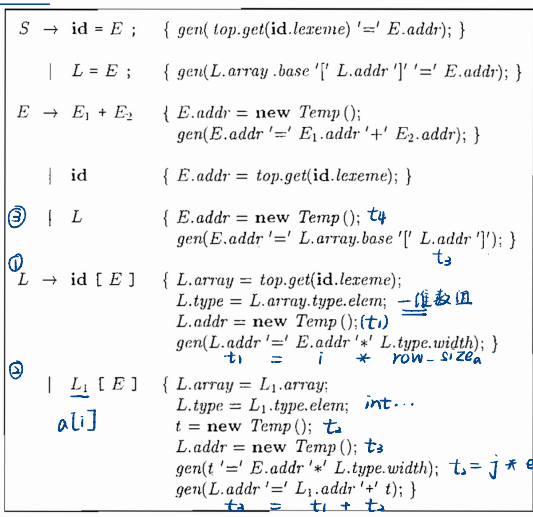


图 6-22 处理数组引用的语义动作

$t_1 = i * row_size_a$

$t_2 = j * ele_size_a$

$t_3 = t_1 + t_2$

$t_4 = a[t_3]$

$t_5 = i * row_size_b$

$t_6 = j * ele_size_b$

$t_7 = t_5 + t_6$

$t_8 = b[t_7]$

$t_9 = t_4 + t_8$

$x = t_9$

练习 6.4.6: 一个按行存放的整数数组 $A[i, j]$ 的下标 i 的范围为 $1 \sim 10$, 下标 j 的范围为 $1 \sim 20$ 。每个整数占 4 个字节。假设数组 A 从 0 字节开始存放, 请给出下列元素的位置:

2) $A[10, 8]$

$9 \times 20 \times 4 + 7 \times 4 = 748$

练习 6.4.8: 一个按行存放的实数型数组 $A[i, j, k]$ 的下标 i 的范围为 $1 \sim 4$, 下标 j 的范围为 $1 \sim 4$, 且下标 k 的范围为 $5 \sim 10$ 。每个实数占 8 个字节。假设数组 A 从 0 字节开始存放。计算下列元素的位置。

2) $A[1, 2, 7]$

$(0 \times 4 \times 8 + 1 \times 8 + 2) \times 8 = 112$

练习 6.5.2: 像 Ada 中那样, 我们假设每个表达式必须具有唯一的类型, 但是我们根据一个子表达式本身只能推导出一个可能类型的集合。也就是说, 将函数 E_1 应用于参数 E_2 (其文法产生式为 $E \rightarrow E_1(E_2)$) 有如下规则:

$E.type = \{\emptyset\}$ 对 $E_2.type$ 中的某个 s , $s \in E_1.type$ 中
描述一个可以确定每个子表达式的唯一类型的语法制导定义 (SDD)。它首先使用属性 $type$, 按照自底向上的方式综合得到一个可能类型的集合。在确定了整个表达式的唯一类型之后, 自顶向下地确定属性 $unique$ 的值, 这个属性表示各个子表达式的类型。

- 本题文法: $S \rightarrow id = E \mid E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id \mid E_1(E_2)$

① 自底向上

产生式	语义规则
$S \rightarrow id = E$	$E.type = \text{symtab}[id].type$
$E \rightarrow E_1 + E_2$	$E.type = \{t \mid \exists t_1 \in E_1.type \wedge \exists t_2 \in E_2.type \wedge t \text{ 在 } (t_1, t_2) \rightarrow t \text{ 上有定义}\}$
$\mid -E_1$	$E.type = \{t \mid \exists t_1 \in E_1.type \wedge - \text{在 } t_1 \rightarrow t \text{ 上有定义}\}$
$\mid (E_1)$	$E.type = E_1.type$
$\mid id$	$E.type = id.type$
$\mid E_1(E_2)$	$E.type = \{t \mid \exists s \in E_2.type \wedge E_1 \text{ 在 } s \rightarrow t \text{ 上有定义}\}$

② 自顶向下

产生式	语义规则
$S \rightarrow id = E$	$E.unique = id.type \text{ if } id.type \in E.type \text{ else error()}$
$E \rightarrow E_1 + E_2$	$\text{if } \exists! (t_1, t_2) \text{ s.t. } t_1 \in E_1.type \wedge t_2 \in E_2.type \wedge t_1 + t_2 \in E.unique : E_1.unique = t_1, E_2.unique = t_2$ else: error()
$\mid -E_1$	(与 $E_1 + E_2$ 类似)
$\mid (E_1)$	$E_1.unique = E.unique$
$\mid id$	$\text{if } E.unique \neq \text{symtab}[id].type : \text{error()}$
$\mid E_1(E_2)$	(与 $E_1 + E_2$ 类似)

练习 6.6.1: 在图 6-36 的语法制导定义中添加处理下列控制流构造的规则:

1) 一个 repeat 语句, **repeat S while B**.

! 2) 一个 for 循环语句, **for (S₁; B; S₂) S₃**.

产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow assign$	$S.code = assign.code$
$S \rightarrow if (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow if (B) S_1 else S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow while (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

图 6-36 控制流语句的语法制导定义

1) 产生式

语义规则

$S \rightarrow repeat S_1 while B$

$B.true: S_1.code$

$S_1.next: B.code$

$B.false$

$S_1.next = new label()$

$B.true = new label()$

$B.false = S.next$

$S.code = label(B.true) \parallel$

$S_1.code \parallel$

$label(S_1.next) \parallel$

$B.code$

2) 产生式

语义规则

$S \rightarrow for (S_1; B; S_2) S_3$

$S_1.next = new label()$

$B.true = new label()$

$B.false = S.next$

$S_2.next = S_1.next$

$S_3.next = new label()$

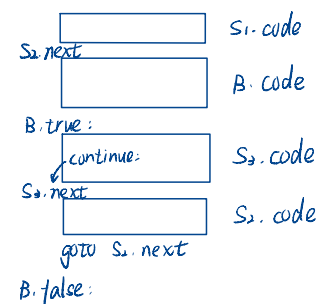
$S.code = S_1.code \parallel label(S_1.next)$

$\parallel B.code \parallel label(B.true)$

$\parallel S_3.code \parallel label(S_3.next)$

$\parallel S_2.code \parallel gen('goto',$

$S_1.next)$



练习 6.7.3: 当使用图 6-46 中的翻译方案对图 6-47 进行翻译时, 我们为每条语句创建 $S.next$ 列表。开始是赋值语句 S_1, S_2, S_3 , 然后逐步处理越来越大的 if 语句、if-else 语句、while 语句和语句块。在图 6-47 中有 5 个这种类型的结构语句:

$S_4: while (E_3) S_1。$

$S_5: if (E_4) S_2。$

$S_6: 包含 S_5 和 S_3 的语句块。$

$S_7: 语句 if (E_2) S_4 else S_6。$

$S_8: 整个程序。$

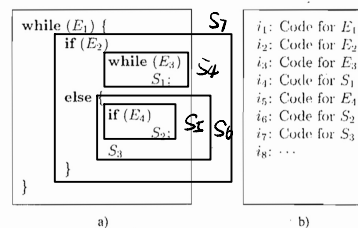


图 6-47 练习 6.7.2 的程序的控制流结构

对于这些结构语句, 我们可以通过一个规则用其他的 $S_j.next$ 列表以及程序中的表达式的列表 $E_k.true$ 和 $E_k.false$ 构造出 $S_i.next$ 。给出计算下列 $next$ 列表的规则:

(1) $S_4.next$ (2) $S_5.next$ (3) $S_6.next$ (4) $S_7.next$ (5) $S_8.next$

(题目有误, 这里应该使用 $nextlist$, $truelist$ 和 $falselist$) 回填

(1) $S_4.nextlist = E_3.falselist$

(2) $S_5.nextlist = E_4.falselist \cup S_1.nextlist$

(3) $S_6.nextlist = S_2.nextlist$ 来自子结点。

(4) $S_7.nextlist = S_4.nextlist \cup S_6.nextlist$

(5) $S_8.nextlist = E_1.falselist$

Ex. 6-1

设有说明

```
int a[5][6][9];
int i, j, k;
```

给出下面语句的三地址中间代码:

```
a[i*2+j, a[2,i+j,k]+i, i*4+k*2] = i*j + k*a[0,4,2*k]
```

```

t1 = i * 2          t16 = t15 + t12
t2 = t1 + j          t17 = t16 * 4
t3 = i + j           t19 = i * j
t4 = t3 * 9           t20 = k * 2
t5 = t4 + 108         t21 = t6 + k * 2
t6 = t5 + k           t22 = t21 * 4
t7 = t6 * 4           t23 = a[t22]
t8 = a[t7]            t24 = k * t23
t9 = t8 + i           t25 = t19 + t24
t10 = i * 4           a[t17] = t25
t11 = k * 2
t12 = t10 + t11
t13 = t2 * 14
t14 = t9 * 9
t15 = t13 + t14

```

Ex. 6-2

给出语句S: if ((a+b*c > m+m) && (x==y)) {while (m>n) m=m-2;} else {a=b+c;} 的三地址中间代码和 S.nextlist。

```

0   t1 = b * c
1   t2 = a + t1
2   t3 = m + m
3   if t2 >= t3 goto 10
4   if x != y goto 10
5   if m >= n goto 13
6   t4 = m - 2
7   m = t4
8   goto 5
9   goto 13
10  t5 = b + c
11  a = t5
12  goto 13

```

S.nextlist = {5, 9, 12}

Ex. 6-3

给出语句S: while ((a+b*c > x+y) && (m==n)) { if (x<=y) { while (a<b) { a=a+10; b=c*m;} } else a=b+c; } 的三地址中间代码和 S.nextlist。

```

1   t1 = b * c
2   t2 = a + t1
3   t3 = x + y
4   if t2 >= t3 goto 18
5   if m != n goto 18
6   if x > y goto 14
7   if a >= b goto 13
8   t4 = a + 10
9   a = t4
10  t5 = c * m
11  b = t5
12  goto 7
13  goto 16
14  t6 = b + c
15  a = t6
16  goto 1
17  goto 18

```

S.nextlist = {4, 5, 17}